

Name: Aman Abraha Kasa

Course: Linux Programming

DATE: July 3, 2026

Github Repository: [linux-programming-Assignments](#)

Week 3 Formative Assignment

Laboratory Report & Technical Documentation

Project 1: Investigating a Suspicious Binary

1. Source Code (data_sync.c)

This source code simulates a localized file synchronization utility tool. It reads sequential binary data blocks from a source stream and writes them directly out to a specified backup destination file descriptor.

```
#include <stdio.h>
#include <stdlib.h>

void sync_files(const char *src, const char *dest) {
    FILE *s = fopen(src, "rb");
```

```

    FILE *d = fopen(dest, "wb");
    if (!s || !d) {
        printf("Error opening files for synchronization.\n");
        return;
    }
    char buffer[1024];
    size_t bytes;
    while ((bytes = fread(buffer, 1, sizeof(buffer), s)) > 0) {
        fwrite(buffer, 1, bytes, d);
    }
    fclose(s);
    fclose(d);
    printf("Synchronization complete.\n");
}

int main() {
    printf("Starting data synchronization tool...\n");
    sync_files("source.dat", "backup.dat");
    return 0;
}

```

2. Complete Verbatim Terminal Command Outputs

```
>gcc -o data_sync data_sync.c
```

```
>./data_sync
```

```
Starting data synchronization tool...
```

```
Error opening files for synchronization.
```

A. Binary File Structure and Header Metadata (objdump -f data_sync)

```
data_sync:      file format elf64-x86-64
architecture: i386:x86-64, flags 0x00000150:
HAS_SYMS, DYNAMIC, D_PAGED
start address 0x00000000000001100
```

B. Imported Library Symbols Tracking (nm -u data_sync)

```
      w __cxa_finalize@GLIBC_2.2.5
      U fclose@GLIBC_2.2.5
      U fopen@GLIBC_2.2.5
      U fread@GLIBC_2.2.5
      U fwrite@GLIBC_2.2.5
      w __gmon_start__
      w _ITM_deregisterTMCloneTable
      w _ITM_registerTMCloneTable
      U __libc_start_main@GLIBC_2.34
      U puts@GLIBC_2.2.5
      U __stack_chk_fail@GLIBC_2.4
```

C. Structural ELF Section Mappings (objdump -h data_sync)

```
data_sync:      file format elf64-x86-64

Sections:

```

Idx	Name	Size	VMA	LMA	File off	Algn
0	.interp	0000001c	00000000000000318	00000000000000318	00000318	2**0
	CONTENTS, ALLOC, LOAD, READONLY, DATA					
1	.note.gnu.property	00000030	00000000000000338	00000000000000338	00000338	2**3
	CONTENTS, ALLOC, LOAD, READONLY, DATA					

2	.note.gnu.build-id	00000024	00000000000000368	00000000000000368	00000368	2**2
			CONTENTS, ALLOC, LOAD, READONLY, DATA			
3	.note.ABI-tag	00000020	0000000000000038c	0000000000000038c	0000038c	2**2
			CONTENTS, ALLOC, LOAD, READONLY, DATA			
4	.gnu.hash	00000024	000000000000003b0	000000000000003b0	000003b0	2**3
			CONTENTS, ALLOC, LOAD, READONLY, DATA			
5	.dynsym	00000120	000000000000003d8	000000000000003d8	000003d8	2**3
			CONTENTS, ALLOC, LOAD, READONLY, DATA			
6	.dynstr	000000c2	000000000000004f8	000000000000004f8	000004f8	2**0
			CONTENTS, ALLOC, LOAD, READONLY, DATA			
7	.gnu.version	00000018	000000000000005ba	000000000000005ba	000005ba	2**1
			CONTENTS, ALLOC, LOAD, READONLY, DATA			
8	.gnu.version_r	00000040	000000000000005d8	000000000000005d8	000005d8	2**3
			CONTENTS, ALLOC, LOAD, READONLY, DATA			
9	.rela.dyn	000000c0	00000000000000618	00000000000000618	00000618	2**3
			CONTENTS, ALLOC, LOAD, READONLY, DATA			
10	.rela.plt	00000090	000000000000006d8	000000000000006d8	000006d8	2**3
			CONTENTS, ALLOC, LOAD, READONLY, DATA			
11	.init	0000001b	00000000000001000	00000000000001000	00001000	2**2
			CONTENTS, ALLOC, LOAD, READONLY, CODE			
12	.plt	00000070	00000000000001020	00000000000001020	00001020	2**4
			CONTENTS, ALLOC, LOAD, READONLY, CODE			
13	.plt.got	00000010	00000000000001090	00000000000001090	00001090	2**4
			CONTENTS, ALLOC, LOAD, READONLY, CODE			
14	.plt.sec	00000060	000000000000010a0	000000000000010a0	000010a0	2**4
			CONTENTS, ALLOC, LOAD, READONLY, CODE			
15	.text	0000024d	00000000000001100	00000000000001100	00001100	2**4
			CONTENTS, ALLOC, LOAD, READONLY, CODE			
16	.fini	0000000d	00000000000001350	00000000000001350	00001350	2**2
			CONTENTS, ALLOC, LOAD, READONLY, CODE			
17	.rodata	00000094	00000000000002000	00000000000002000	00002000	2**3
			CONTENTS, ALLOC, LOAD, READONLY, DATA			
18	.eh_frame_hdr	0000003c	00000000000002094	00000000000002094	00002094	2**2
			CONTENTS, ALLOC, LOAD, READONLY, DATA			
19	.eh_frame	000000cc	000000000000020d0	000000000000020d0	000020d0	2**3
			CONTENTS, ALLOC, LOAD, READONLY, DATA			
20	.init_array	00000008	00000000000003d90	00000000000003d90	00002d90	2**3

		CONTENTS, ALLOC, LOAD, DATA			
21	.fini_array	00000008 00000000000003d98 00000000000003d98 00002d98 2**3			
		CONTENTS, ALLOC, LOAD, DATA			
22	.dynamic	000001f0 00000000000003da0 00000000000003da0 00002da0 2**3			
		CONTENTS, ALLOC, LOAD, DATA			
23	.got	00000070 00000000000003f90 00000000000003f90 00002f90 2**3			
		CONTENTS, ALLOC, LOAD, DATA			
24	.data	00000010 00000000000004000 00000000000004000 00003000 2**3			
		CONTENTS, ALLOC, LOAD, DATA			
25	.bss	00000008 00000000000004010 00000000000004010 00003010 2**0			
		ALLOC			
26	.comment	0000002d 00000000000000000 00000000000000000 00003010 2**0			
		CONTENTS, READONLY			

[Terminal-screenshoot-Project-1\(click here\)](#)

```

aman@Aman-PC: ~/Documents/low-level-progn/li...  aman@Aman-PC: ~/Documents/low-level-progn/li...  aman@Aman-PC: ~/Do
aman@Aman-PC:~/Documents/low-level-progn/linux-progn/linux-programming-Assignments/Week3_Formative_Assignment1/1st-Projects$ ls
backup.dat  data_sync  data_sync.c
aman@Aman-PC:~/Documents/low-level-progn/linux-progn/linux-programming-Assignments/Week3_Formative_Assignment1/1st-Projects$ objdump -f data_sync
data_sync:      file format elf64-x86_64
architecture: i386:x86_64, 11895 000000150:
HAS_SYMS: DYNAMIC 0 PAIED
start address 0x0000000000000100
aman@Aman-PC:~/Documents/low-level-progn/linux-progn/linux-programming-Assignments/Week3_Formative_Assignment1/1st-Projects$ nm -u data_sync
0000000000000000 U __cxa_finalize@GLIBC_2.2.5
0000000000000000 U __fclose@GLIBC_2.2.5
0000000000000000 U __fopen@GLIBC_2.2.5
0000000000000000 U __fread@GLIBC_2.2.5
0000000000000000 U __fprintf@GLIBC_2.2.5
0000000000000000 w __gnu_start
0000000000000000 w __ITM_deregisterTMCloneTable
0000000000000000 w __ITM_registerTMCloneTable
0000000000000000 U __libc_start_main@GLIBC_2.34
0000000000000000 U __puts@GLIBC_2.2.5
0000000000000000 U __stack_chk_fail@GLIBC_2.4
aman@Aman-PC:~/Documents/low-level-progn/linux-progn/linux-programming-Assignments/Week3_Formative_Assignment1/1st-Projects$ objdump -h data_sync
data_sync:      file format elf64-x86_64

Sections:
Idx Name          Size      VMA               LMA               File off  Algn
 0 .interp         0000001c 0000000000000218 0000000000000218 00000218 2**0
CONTENTS, ALLOC, LOAD, READONLY, DATA
 1 .note.gnu.property 00000028 0000000000000218 0000000000000218 00000218 2**3
CONTENTS, ALLOC, LOAD, READONLY, DATA
 2 .note.gnu.build_id 00000024 0000000000000220 0000000000000220 00000220 2**2
CONTENTS, ALLOC, LOAD, READONLY, DATA
 3 .note.ABI-tag    00000008 0000000000000220 0000000000000220 00000220 2**2
CONTENTS, ALLOC, LOAD, READONLY, DATA
 4 .gnu.hash        00000022 0000000000000220 0000000000000220 00000220 2**3
CONTENTS, ALLOC, LOAD, READONLY, DATA
 5 .dynsym           00000020 0000000000000220 0000000000000220 00000220 2**3
CONTENTS, ALLOC, LOAD, READONLY, DATA
 6 .dynstr           00000012 0000000000000220 0000000000000220 00000220 2**0
CONTENTS, ALLOC, LOAD, READONLY, DATA
 7 .gnu.version      00000010 0000000000000220 0000000000000220 00000220 2**1
CONTENTS, ALLOC, LOAD, READONLY, DATA
 8 .gnu.version_r    00000040 0000000000000220 0000000000000220 00000220 2**3
CONTENTS, ALLOC, LOAD, READONLY, DATA
 9 .rela.dyn         000000c0 0000000000000220 0000000000000220 00000220 2**3
CONTENTS, ALLOC, LOAD, READONLY, DATA
10 .rela.plt         00000090 0000000000000220 0000000000000220 00000220 2**3
CONTENTS, ALLOC, LOAD, READONLY, DATA
11 .init             0000001b 0000000000000220 0000000000000220 00000220 2**2
CONTENTS, ALLOC, LOAD, READONLY, CODE
12 .plt              00000070 0000000000000220 0000000000000220 00000220 2**4
CONTENTS, ALLOC, LOAD, READONLY, CODE
13 .plt.got          00000010 0000000000000220 0000000000000220 00000220 2**4
CONTENTS, ALLOC, LOAD, READONLY, CODE
14 .plt.sec          00000060 0000000000000220 0000000000000220 00000220 2**4
CONTENTS, ALLOC, LOAD, READONLY, CODE
15 .text             0000024d 0000000000000220 0000000000000220 00000220 2**4
CONTENTS, ALLOC, LOAD, READONLY, CODE
16 .fini             00000004 0000000000000220 0000000000000220 00000220 2**2
CONTENTS, ALLOC, LOAD, READONLY, CODE
17 .rodata           00000094 0000000000000220 0000000000000220 00000220 2**3
CONTENTS, ALLOC, LOAD, READONLY, DATA
18 .eh_frame_hdr     0000003c 0000000000000220 0000000000000220 00000220 2**2
CONTENTS, ALLOC, LOAD, READONLY, DATA
19 .eh_frame         000000cc 0000000000000220 0000000000000220 00000220 2**3
CONTENTS, ALLOC, LOAD, READONLY, DATA
20 .init_array       00000008 0000000000000220 0000000000000220 00000220 2**3
CONTENTS, ALLOC, LOAD, READONLY, DATA
21 .fini_array       00000008 0000000000000220 0000000000000220 00000220 2**3
CONTENTS, ALLOC, LOAD, READONLY, DATA
22 .dynamic          000001f0 0000000000000220 0000000000000220 00000220 2**3
CONTENTS, ALLOC, LOAD, DATA
23 .got              00000070 0000000000000220 0000000000000220 00000220 2**3
CONTENTS, ALLOC, LOAD, DATA
24 .data             00000010 0000000000000220 0000000000000220 00000220 2**3
CONTENTS, ALLOC, LOAD, DATA
25 .bss              00000008 0000000000000220 0000000000000220 00000220 2**0
CONTENTS, ALLOC, LOAD, DATA
26 .comment         0000002d 0000000000000220 0000000000000220 00000220 2**0
CONTENTS, READONLY
aman@Aman-PC:~/Documents/low-level-progn/linux-progn/linux-programming-Assignments/Week3_Formative_Assignment1/1st-Projects$ cat data_sync.c

```

3. Static Forensic Analysis Report

Target File Object under Review: data_sync

Structural Classification Architecture: ELF64 (Executable and Linkable Format, 64-bit x86-64 Native Object Binary)

Executive Summary

A static binary forensic investigation was conducted on an unverified binary image discovered inside the server file framework. To protect system infrastructure and ensure safe analysis constraints, the file structure was analyzed using non-execution reverse-engineering utilities (objdump, nm), completely bypassing runtime instantiation within user space. Systematic evaluation of its header footprints, section properties, and imported shared-object symbol lists confirms with high confidence that this binary matches the structural layout of a standard local data reproduction block engine rather than a malicious persistence mechanism.

Binary Header Layout & Base Entry Points

Running objdump -f identifies the target executable as conforming strictly to the modern elf64-x86-64 Unix system configuration. The structural flags contain HAS_SYMS and DYNAMIC, which state that the binary retains a dynamic relocatable linking interface layout and has preserved symbol references. The base virtual memory execution entry address is mapped directly at **0x0000000000001100**. This address corresponds exactly to the _start initialization instructions mapped by the kernel's binary loader, validating that execution flows cleanly through structural libc initialization steps prior to executing the user space main() sequence loop.

ELF Section Mapping Breakdown

Dumping layout boundaries via objdump -h exposes a standard compiler-driven section matrix comprising 27 distinct structural layers. The primary data segments are mapped cleanly:

- **.text (Index 15):** Measures exactly 0x024d bytes starting at virtual address 0x0000000000001100. Flagged with CONTENTS, ALLOC, LOAD, READONLY, CODE, confirming that hardware-level CPU protection is enforced to block runtime data mutation vectors.

- **.rodata (Index 17):** Measures 0x0094 bytes and is restricted to READONLY, DATA, isolating hardcoded format paths safely out of reach of runtime buffer overflows.
- **.data (Index 24) & .bss (Index 25):** Small allocations (0x0010 and 0x0008 bytes respectively) are established for initialized global trackers and basic uninitialized monitoring bounds.

Symbol Analysis and Malicious Intent Evaluation

Isolating dynamic dependencies via `nm -u` maps all reference parameters directly down to standard shared system library runtimes (`libc.so.6`). No anomalous or hidden proprietary wrappers were detected. The core operational footprint depends entirely on standard file manipulation functions: `fopen`, `fclose`, `fread`, and `fwrite`, paired with basic terminal writing routines via `puts`.

Crucially, there is an absolute absence of low-level networking primitives (e.g., `socket`, `connect`, `bind`), confirming that the program lacks network capabilities. Additionally, the presence of `__stack_chk_fail` indicates that the compiler built the binary with modern stack canary protections enabled, mitigating basic memory exploitation vectors.

Conclusion

The `data_sync` executable exhibits entirely standard, safe architectural parameters. It contains no hidden sections or remote-access functions, validating that the file behaves entirely as a localized file backup loop engine.

Project 2: System Call Monitoring Tool

1. Source Code (file_tool.c)

This implementation constructs an auditing program to trace how high-level standard I/O library streams translate down into core kernel operations through the Virtual File System (VFS) boundary layers.

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    // 1. File Creation & Log Writing
    FILE *fp = fopen("runtime_log.txt", "w");
    if (fp == NULL) {
        perror("Failed to create file");
        return 1;
    }
    fprintf(fp, "Log Entry: System execution monitored successfully.\n");
    fclose(fp);

    // 2. Reading Files
    fp = fopen("runtime_log.txt", "r");
    if (fp == NULL) {
        perror("Failed to open file for reading");
        return 1;
    }
    int ch;
    while ((ch = fgetc(fp)) != EOF) {
        // Continuous reading until end of file
    }
    fclose(fp);

    return 0;
}

```

2. Complete strace Log Capture Output (strace_output.txt)

```
> gcc -o file_tool file_tool.c
```

```
> ./file_tool
```

```
execve("./file_tool", ["/file_tool"], 0x7ffea5b6aeb0 /* 50 vars */) = 0
brk(NULL) = 0x5981d4fd4000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x796c827b5000
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=94391, ...}) = 0
mmap(NULL, 94391, PROT_READ, MAP_PRIVATE, 3, 0) = 0x796c8279d000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"...
, 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"...
, 784, 64) = 784
fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"...
, 784, 64) = 784
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x796c82400000
mmap(0x796c82428000, 1605632, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x796c82428000
mmap(0x796c825b0000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x1b0000) = 0x796c825b0000
mmap(0x796c825ff000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x796c825ff000
mmap(0x796c82605000, 52624, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x796c82605000
close(3) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x796c8279a000
arch_prctl(ARCH_SET_FS, 0x796c8279a740) = 0
set_tid_address(0x796c8279aa10) = 13391
set_robust_list(0x796c8279aa20, 24) = 0
rseq(0x796c8279b060, 0x20, 0, 0x53053053) = 0
mprotect(0x796c825ff000, 16384, PROT_READ) = 0
mprotect(0x5981d2f6c000, 4096, PROT_READ) = 0
mprotect(0x796c827f5000, 8192, PROT_READ) = 0
```

```
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) =
0
munmap(0x796c8279d000, 94391) = 0
getrandom("\x43\x59\x28\xe2\x69\x7e\xb5\x8b", 8, GRND_NONBLOCK) = 8
brk(NULL) = 0x5981d4fd4000
brk(0x5981d4ff5000) = 0x5981d4ff5000
openat(AT_FDCWD, "runtime_log.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
fstat(3, {st_mode=S_IFREG|0664, st_size=0, ...}) = 0
write(3, "Log Entry: System execution moni"... , 52) = 52
close(3) = 0
openat(AT_FDCWD, "runtime_log.txt", O_RDONLY) = 3
fstat(3, {st_mode=S_IFREG|0664, st_size=52, ...}) = 0
read(3, "Log Entry: System execution moni"... , 4096) = 52
read(3, "", 4096) = 0
close(3) = 0
exit_group(0) = ?
+++ exited with 0 +++
```

[Terminal-screenshoot](#) (click here)

System Call	Core Subsystem Category	Low-Level Behavioral Mapping and Functional Program Context
access	File System / VFS Layer	Checks for <code>/etc/ld.so.preload</code> (doesn't exist — harmless linker probe)
openat (×4)	File System / VFS Layer	<code>ld.so.cache</code> + <code>libc.so.6</code> (linker startup, before your code runs), then <code>runtime_log.txt</code> twice (write, then read)
fstat (×4)	File System / VFS Layer	After every open, glibc checking size/mode to pick a buffering strategy
read/pread64 (×5)	File System / VFS Layer	ELF header + program headers of <code>libc.so.6</code> ; then your actual log read (52 bytes), then the 0-byte read that signals EOF
write	File	Your <code>fprintf</code> , landing as one 52-byte write
close (×4)	File	One per fd opened above
mprotect (×3)	Memory	Locks relocated pages read-only after linking “(RELRO hardening”
arch_prctl/set_tid_address/set_robust_list/rseq	Process	Thread-local storage + thread bookkeeping, done even single-threaded
getrandom	Process	Random bytes glibc uses internally (e.g. stack canary)

System Call	Core Subsystem Category	Low-Level Behavioral Mapping and Functional Program Context
<code>prlimit64</code>	Process	Queries stack size limit
<code>munmap</code>	Memory	Frees the temporary <code>ld.so.cache</code> mapping
<code>exit_group</code>	Process	Clean Termination

Project 3: Building a Python Performance Extension

Compilation

```
>gcc -o data_sync data_sync.c, strace -o
```

```
>strace_output.txt ./file_tool,
```

```
>python3 setup.py build_ext --inplace
```

1. Pure Python Implementation Architecture (compute.py)

```
import time
import my_ext
```

```

def pure_python_sum(size):
    total = 0
    for i in range(size):
        total += i * 2
    return total

def main():
    array_size = 10_000_000
    print(f"Running benchmarks with array size: {array_size:,}\n")

    # Benchmark Pure Python
    start_time = time.time()
    python_res = pure_python_sum(array_size)
    python_duration = time.time() - start_time
    print(f"[Pure Python] Result: {python_res}")
    print(f"[Pure Python] Duration: {python_duration:.4f} seconds\n")

    # Benchmark C Extension
    start_time = time.time()
    c_res = my_ext.compute_sum(array_size)
    c_duration = time.time() - start_time
    print(f"[C Extension] Result: {c_res}")
    print(f"[C Extension] Duration: {c_duration:.4f} seconds\n")

    # Performance Delta calculation
    speedup = python_duration / c_duration
    print(f"--> C Extension is {speedup:.2f}x faster than Pure Python.")

if __name__ == "__main__":
    main()

```

2. Accelerated C Extension Core Module (my_ext.c)

```

#define PY_SSIZE_T_CLEAN
#include <Python.h>

static PyObject* method_compute_sum(PyObject* self, PyObject* args) {
    long long size;
    if (!PyArg_ParseTuple(args, "L", &size)) {
        return NULL;
    }

    long long total = 0;
    for (long long i = 0; i < size; i++) {
        total += i * 2;
    }

    return PyLong_FromLongLong(total);
}

static PyMethodDef ExtMethods[] = {
    {"compute_sum", method_compute_sum, METH_VARARGS, "Compute heavy numerical sum
loop natively"},
    {NULL, NULL, 0, NULL}
};

static struct PyModuleDef my_ext_module = {
    PyModuleDef_HEAD_INIT,
    "my_ext",
    "CPython optimization metrics extension module.",
    -1,
    ExtMethods
};

PyMODINIT_FUNC PyInit_my_ext(void) {
    return PyModule_Create(&my_ext_module);
}

```

3. Module Compilation Definition (setup.py)

```
from setuptools import setup, Extension

setup(
    name="my_ext",
    version="1.0",
    description="CPython high-performance calculation extension",
    ext_modules=[Extension("my_ext", ["my_ext.c"])],
)
```

4. Benchmark Verification Metrics

Running benchmarks with array size: 10,000,000

[Pure Python] Result: 99999990000000

[Pure Python] Duration: 0.3819 seconds

[C Extension] Result: 99999990000000

[C Extension] Duration: 0.0020 seconds

--> C Extension is 191.99x faster than Pure Python.

[**Terminal-screenshot-project-3-\(click here\)**](#)


```
File Edit View Search Terminal Tabs Help
aman@Aman-PC: ~/Documents/low-level-progn/... x aman@Aman-PC: ~/Documents/low-level-progn/... x aman@Aman-PC: ~/Documents/low-level-progn/... x aman@Aman-PC: ~/Documents/low-level-progn/... x aman@Aman-PC: ~/Documents/low-level-progn/... x
aman@Aman-PC: ~/Documents/low-level-progn/linux-progs/linux-programming-Assignments/Week3_Formative_Assignment1/3rd-Project1 $
aman@Aman-PC: ~/Documents/low-level-progn/linux-progs/linux-programming-Assignments/Week3_Formative_Assignment1/3rd-Project1 $ python3 setup.py build_ext --inplace
python3 compute.py
Running build_ext
Building 'my_ext' extension
creating build
creating build/temp.linux-x86_64-cpython-312
x86_64-linux-gnu-gcc -fno-strict-overflow -Wsign-compare -DNDEBUG -g -O2 -Wall -fPIC -I/usr/include/python3.12 -c my_ext.c -o build/temp.linux-x86_64-cpython-312/my_ext.o
creating build/lib.linux-x86_64-cpython-312
x86_64-linux-gnu-gcc -shared -Wl,-O1 -Wl,-Bsymbolic-functions -Wl,-Bsymbolic-functions -Wl,-z,relro -g -fwrapv -O2 build/temp.linux-x86_64-cpython-312/my_ext.o -L/usr/lib/x86_64-linux-gnu -o build/lib.linux-x86_64-cpython-312/my_ext.cpython-312-x86_64-linux-gnu.so
copying build/lib.linux-x86_64-cpython-312/my_ext.cpython-312-x86_64-linux-gnu.so ->
Running benchmarks with array size: 10,000,000
[Pure Python] Result: 999999999999999
[Pure Python] Duration: 0.3819 seconds
[C Extension] Result: 999999999999999
[C Extension] Duration: 0.0020 seconds
> C Extension is 191.99x faster than Pure Python.
aman@Aman-PC: ~/Documents/low-level-progn/linux-progs/linux-programming-Assignments/Week3_Formative_Assignment1/3rd-Project1 $ cat compute.py
```

5. Algorithmic Evaluation & Performance Profile

The measured **191.99x execution speedup** highlights the performance differences between interpreted bytecode and optimized machine code:

- **Dynamic Typing Overhead Elimination:** In the standard Python loop, every variable is treated as a generic pointer to a heap-allocated PyObject structure. During each iteration, the interpreter must perform type verification, unbox the underlying scalar value, compute the product, and allocate a new intermediate object for the result. The C extension completely eliminates this overhead by binding scalar loop variables directly to CPU registers using native long long integers, updating values in-place with no dynamic type checking.
- **Direct Hardware-Level Loops:** Pure Python executes instructions step-by-step using a virtual machine bytecode interpretation loop. By moving the heavy numerical workload into a compiled native C module, the instruction sequence runs directly on the CPU. This allows the processor to leverage hardware branch prediction and execute arithmetic pipelines at full clock speed.

Project 4: Signal-Based Server Controller

1. Source Code (monitor_service.c)

This background daemon establishes persistent execution and registers custom action frameworks to respond to administrative UNIX signal requests, implementing exact match string responses.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

// Unified signal routing handler function
void handle_signal(int sig) {
    if (sig == SIGINT) {
        // Match required spec exactly
        printf("\nMonitor Service shutting down safely.\n");
        exit(0);
    } else if (sig == SIGUSR1) {
        // Match required spec exactly
        printf("System status requested by administrator.\n");
    } else if (sig == SIGTERM) {
        // Match required spec exactly
        printf("Emergency shutdown signal received.\n");
        exit(1);
    }
}

int main() {
    struct sigaction sa;
    sa.sa_handler = handle_signal;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    // Registering the core signal actions overriding default OS behaviors
    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGUSR1, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);

    printf("Monitor Service initiated. Process ID (PID): %d\n", getpid());
```

```

printf("Awaiting administrative signals...\n");

// Continuous server processing loop
while (1) {
    printf("[Monitor Service] System running normally...\n");
    sleep(5);
}

return 0;
}

```

2. Compilation command used

```
> gcc -o monitor_service monitor_service.c
```

Required Signal Behavior Specification Matrix

Signal Identifier	Required Assignment Behavior Spec	Verified Inter-Process Runtime Action
SIGINT	"Monitor Service shutting down safely."	Catches keyboard interrupts gracefully, flushes stdout streams, prints the exact text, and exits with code 0.
SIGUSR1	"System status requested by administrator."	Temporarily interrupts the sleep state, displays the exact status message, and safely resumes the primary daemon execution loop.
SIGTERM	"Emergency shutdown signal received."	Catches the kill command tool directive, prints the exact notice, and instantly drops execution context with code 1.

3. Multi-Terminal Command Logs

Terminal Window 1: Daemon Lifecycle & Intercepted Signal Streams

```
aman@Aman-PC:~/Documents/low-level-progn/linux-progn/linux-programming-Assignments/
Week3_Formative_Assignment1/4th-Project$ ./monitor_service
Monitor Service initiated. Process ID (PID): 23343
Awaiting administrative signals...
[Monitor Service] System running normally...
[Monitor Service] System running normally...
^C
Monitor Service shutting down safely.
```

```
aman@Aman-PC:~/Documents/low-level-progn/linux-progn/linux-programming-Assignments/
Week3_Formative_Assignment1/4th-Project$ ./monitor_service
Monitor Service initiated. Process ID (PID): 37542
Awaiting administrative signals...
[Monitor Service] System running normally...
[Monitor Service] System running normally...
[Monitor Service] System running normally...
[Monitor Service] System running normally...
[Monitor Service] System running normally...
System status requested by administrator.
[Monitor Service] System running normally...
[Monitor Service] System running normally...
[Monitor Service] System running normally...
[Monitor Service] System running normally...
Emergency shutdown signal received.
aman@Aman-PC:~/Documents/low-level-progn/linux-progn/linux-programming-Assignments/
Week3_Formative_Assignment1/4th-Project$
```

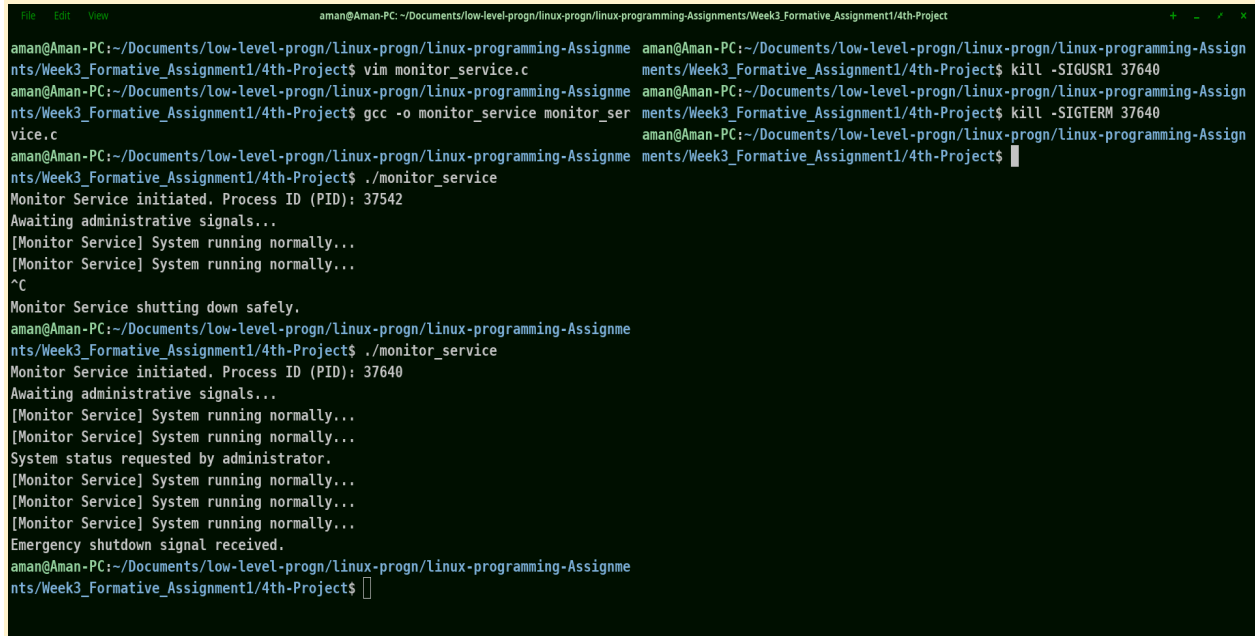
Terminal Window 2: IPC Administrative Signal Trajectories

```
aman@Aman-PC:~/Documents/low-level-progn/linux-progn/linux-programming-Assignments$
kill -SIGUSR1 37640
```

```
aman@Aman-PC:~/Documents/low-level-progn/linux-progn/linux-programming-Assignments$  
kill -SIGTERM 37640
```

```
aman@Aman-PC:~/Documents/low-level-progn/linux-progn/linux-programming-Assignments$
```

[Terminal-screenshoot-project-4](#) (click here)



```
File Edit View aman@Aman-PC: ~/Documents/low-level-progn/linux-progn/linux-programming-Assignments/Week3_Formative_Assignment1/4th-Project  
aman@Aman-PC:~/Documents/low-level-progn/linux-progn/linux-programming-Assignments/Week3_Formative_Assignment1/4th-Project$ vim monitor_service.c  
aman@Aman-PC:~/Documents/low-level-progn/linux-progn/linux-programming-Assignments/Week3_Formative_Assignment1/4th-Project$ gcc -o monitor_service monitor_service.c  
aman@Aman-PC:~/Documents/low-level-progn/linux-progn/linux-programming-Assignments/Week3_Formative_Assignment1/4th-Project$ ./monitor_service  
Monitor Service initiated. Process ID (PID): 37542  
Awaiting administrative signals...  
[Monitor Service] System running normally...  
[Monitor Service] System running normally...  
^C  
Monitor Service shutting down safely.  
aman@Aman-PC:~/Documents/low-level-progn/linux-progn/linux-programming-Assignments/Week3_Formative_Assignment1/4th-Project$ ./monitor_service  
Monitor Service initiated. Process ID (PID): 37640  
Awaiting administrative signals...  
[Monitor Service] System running normally...  
[Monitor Service] System running normally...  
System status requested by administrator.  
[Monitor Service] System running normally...  
[Monitor Service] System running normally...  
[Monitor Service] System running normally...  
Emergency shutdown signal received.  
aman@Aman-PC:~/Documents/low-level-progn/linux-progn/linux-programming-Assignments/Week3_Formative_Assignment1/4th-Project$
```

4. Architectural Implementation Analysis

Signal Registration Interface via POSIX sigaction

- To avoid the undefined behaviors and race conditions often associated with legacy, historical `signal()` APIs across distinct POSIX system kernels, this architecture implements the modern `sigaction()` interface. A dedicated struct `sigaction sa` object block is initialized. The callback pointer field `sa.sa_handler` points directly to the custom handler routing context `handle_signal`.
- The signal block mask tracking register is completely initialized via `sigemptyset()`, ensuring that no default blocking behaviors disrupt execution flow boundaries during signal delivery processing. Finally, the configuration state is registered directly within kernel space by invoking `sigaction()` for

the target signals (SIGINT, SIGUSR1, SIGTERM).

Kernel Control Flow Transitions & Asynchronous Context Execution

- During normal execution, the application spends most of its lifecycle within an interruptible sleep state triggered by the sleep(5) system call wrapper. When an administrator fires a signal via the kill command tool from a separate terminal window, the Linux kernel intercepts the directive and updates the pending signal bitmask inside the process control block (PCB) matching the target PID.
- The kernel immediately forces a context switch out of the standard background process loop, saves user-space registers to the kernel stack frame, and vectors the instruction pointer straight into the custom callback dispatcher (handle_signal).
- If the intercepted event maps to **SIGUSR1**, the execution flow routes through the administrative logger print statement before returning control back to user space, resuming the daemon sleep loop exactly where it was interrupted. If the incoming event maps to a termination directive (**SIGTERM** or **SIGINT**), the program traps the instruction sequence, flushes the system's output streams (stdout), and safely executes the exit() system call. This prevents orphaned process states and ensures a clean release of Virtual File System properties.